- CPSC 375: High-Performance Computing

CPSC 375: High-Performance Computing

Spring 2026

- Syllabus
- Schedule
- Upload
- The Cluster Project

Homework 19

NOTE: You are not required to hand in the following exercises, but you are strongly encouraged to complete them to strengthen your understanding of the concepts covered in class.

1. This exercise demonstrates why `atomic` is preferred for simple updates. Simulate 100,000 small deposits of $1 to a single `int balance = 0`.
   A. Run the loop using `#pragma omp critical`.
   B. Run the loop using `#pragma omp atomic`.
   Use `omp_get_wtime()` to time both versions. Which one is faster? Why is `atomic` able to outperform `critical` for a simple increment?

2. Static scheduling is great for equal work, but terrible for unequal work. Write a loop from `i` = 0 to 20. Inside the loop, make the thread sleep or do dummy work proportional to `i`.
   A. Run with `schedule(static, 1)`.
   B. Run with `schedule(dynamic, 1)`.
   Print which thread handles which iteration `i`. In the `static` version, do the threads with the higher `i` values finish much later than the others? How does `dynamic` fix this?

3. Not all parallelism is a loop. Sometimes you want to do two completely different things at once. Create a program that performs two independent math operations on a large dataset:
   - Section 1: Calculate the average of the array.
   - Section 2: Find the minimum value of the array.

   Wrap these two distinct functions in `#pragma omp parallel sections`. Question: If you have 4 threads available but only 2 sections defined, what do the other 2 threads do?

- **Welcome: Sean**
  - LogOut

Trinity College
HARTFORD CONNECTICUT